

Curso de Go

Aula 14 - Gerenciamento de dependências em Go

Professor :Antônio Marcelo

NÚCLEA
ACADEMY

 **códigos_**^[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **O que é o go.mod**
- **Como iniciar um projeto com go mod init**
- **Como adicionar dependências**
- **Como atualizar versões**
- **Como funciona o versionamento semântico**
- **Como usar replace para testar módulos locais**
- **Como evitar problemas comuns em projetos reais**
- **Parte Prática**



O que é gerenciamento de dependências

- Dependências são pacotes externos que o seu projeto usa.
- Exemplo: uma API pode usar um pacote para criar rotas HTTP, outro para logs e outro para acessar banco de dados.
- Sem gerenciamento de dependências, o projeto vira uma bagunça, pois cada computador pode usar uma versão diferente da biblioteca.
- Em Go, o gerenciamento moderno é feito com Go Modules.
- Arquivos principais:

```
go.mod  
go.sum
```

- O go.mod define o nome do módulo e suas dependências.
- O go.sum registra verificações de segurança das versões baixadas.



Criando um módulo com go mod init

- **Todo projeto Go moderno deve começar com:**

```
go mod init nome-do-modulo
```

- **Exemplo:**

```
go mod init github.com/antonio/produtos
```

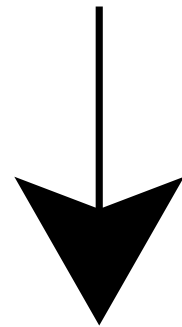
- **Isso cria o diretório “produtos”**
- **O nome do módulo normalmente segue o caminho do repositório.**
- **Esse nome será usado nos imports internos do projeto.**



Estrutura inicial de um projeto Go com módulo

- Exemplo de organização:

```
produtos/  
go.mod  
go.sum  
main.go  
internal/  
estoque/  
estoque.go
```



Estoque.go

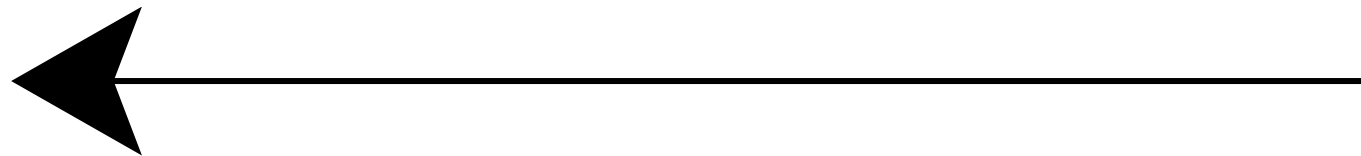
```
package estoque
```

```
func CalcularTotal(qtd int, preco float64) float64 {  
    return float64(qtd) * preco  
}  
}
```

Main.go

```
package main
```

```
import (  
    "fmt"  
  
    "github.com/antonio/produtos/internal/estoque"  
)  
  
func main() {  
    total := estoque.CalcularTotal(10, 25.50)  
    fmt.Println("Total:", total)  
}
```

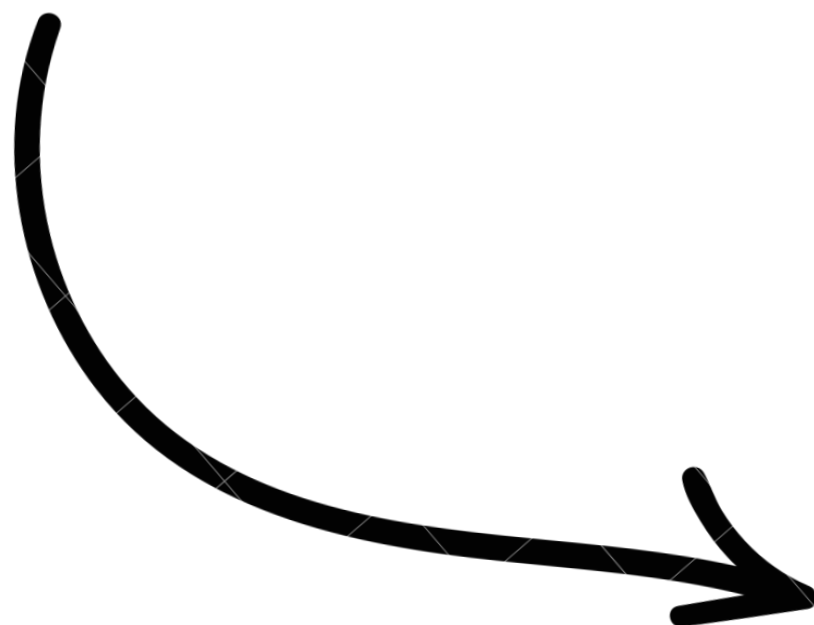




Adicionando uma dependência externa

- Quando usamos uma biblioteca externa, o Go pode adicioná-la automaticamente.
- Exemplo usando o pacote chi para rotas HTTP:

```
go get github.com/go-chi/chi/v5  
require github.com/go-chi/chi/v5 v5.0.12
```



```
package main  
  
import (  
    "net/http"  
  
    "github.com/go-chi/chi/v5"  
)  
  
func main() {  
    r := chi.NewRouter()  
  
    r.Get("/", func(w  
http.ResponseWriter, r *http.Request)  
{  
    w.Write([]byte("API funcionando"))  
    })  
  
    http.ListenAndServe(":8080", r)  
}
```



O papel do go.sum

- O go.sum não é uma lista simples de dependências.
- Ele guarda hashes das versões usadas pelo projeto.
- Isso ajuda a garantir que a dependência baixada é exatamente a mesma que foi usada antes.
- Exemplo:

```
github.com/go-chi/chi/v5 v5.0.12 h1:...
```

```
github.com/go-chi/chi/v5 v5.0.12/go.mod h1:...
```

- Na prática:
 - O go.mod diz quais dependências o projeto usa
 - O go.sum ajuda a validar a integridade dessas dependências
 - Ambos devem ser versionados no Git
- O comando “go mod tidy “ remove dependências não usadas e adiciona as que faltam.



Versionamento semântico

- **Go usa versões no formato semântico:**

`vMAJOR.MINOR.PATCH`

Exemplo

`v1.4.2`

- **Significado:**
 - **MAJOR:** mudança grande, pode quebrar compatibilidade
 - **MINOR:** nova funcionalidade, sem quebrar compatibilidade
 - **PATCH:** correção de bug

`v1.0.0`

`v1.1.0`

`v1.1.1`

`v2.0.0`

- **Uma mudança de v1 para v2 indica que algo importante pode ter mudado na API.**



Regra especial para versões v2 ou superiores

- Em Go, módulos com versão principal v2 ou superior normalmente precisam indicar a versão no caminho do import.
- Exemplo:

```
import "github.com/go-chi/chi/v5"
```

- Perceba o /v5 no final. Isso acontece porque o Go trata versões principais diferentes como módulos diferentes.
- Exemplo conceitual:

```
github.com/exemplo/lib/v1  
github.com/exemplo/lib/v2  
github.com/exemplo/lib/v3
```

- Isso evita que uma atualização grande quebre seu projeto sem controle.
- A documentação oficial explica que versões v2 ou maiores devem ter o sufixo da versão principal no caminho do módulo.



Atualizando e controlando versões

- Para ver módulos disponíveis:

```
go list -m -versions github.com/go-chi/chi/v5
```

- Para atualizar uma dependência:

```
go get github.com/go-chi/chi/v5@latest
```

- Para usar uma versão específica:

```
go get github.com/go-chi/chi/v5@v5.0.12
```

- Para atualizar todas as dependências possíveis:

```
go get -u ./...
```

Importante!!

- Nunca atualize dependências sem testar
- Leia changelogs quando mudar versão major
- Faça commit separado para atualizações importantes



Usando replace com dependências locais

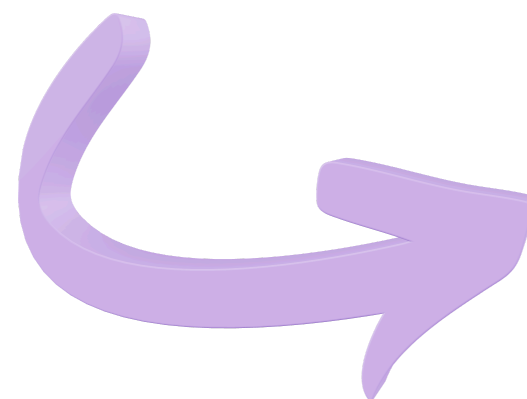
- O replace permite trocar uma dependência por outra origem.
- Muito usado quando estamos desenvolvendo dois módulos ao mesmo tempo.

```
meu-sistema/  
  api/  
  go.mod  
biblioteca/  
  go.mod
```

- Exemplo de replace no módulo go.mod

```
require github.com/antonio/biblioteca v0.0.0
```

```
replace github.com/antonio/biblioteca =>  
../biblioteca
```



Agora, quando a API importar:

import "github.com/antonio/biblioteca"

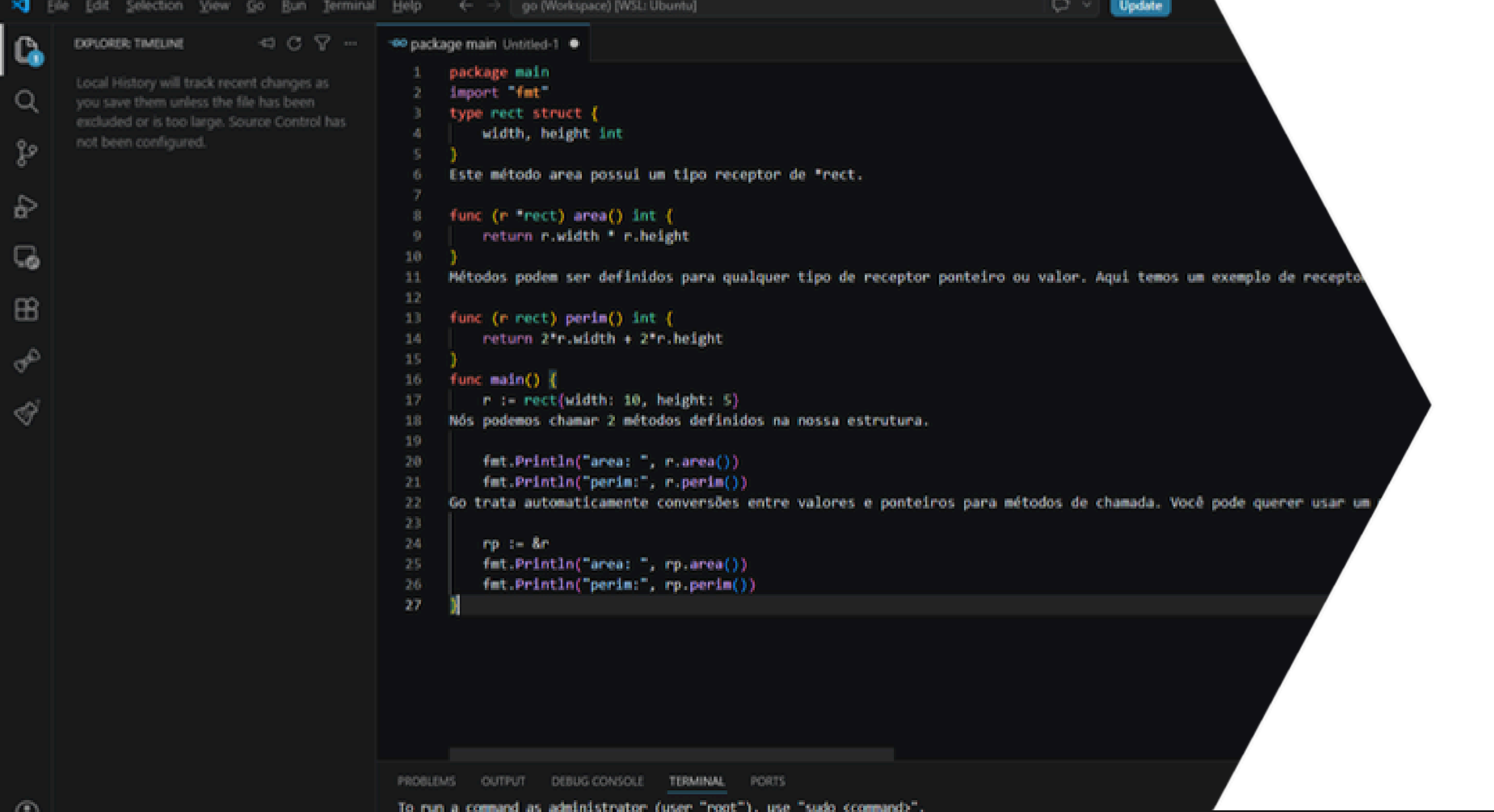
O Go usará a pasta local:

../biblioteca



Boas práticas

- **Sempre use go mod init em novos projetos**
- **Versione go.mod e go.sum no Git**
- **Use go mod tidy com frequência**
- **Evite replace em produção**
- **Use replace para testes locais**
- **Trave versões importantes quando precisar de estabilidade**
- **Teste o projeto após atualizar dependências**



The screenshot shows a Go IDE with a package main. The code defines a struct `rect` with `width` and `height` fields. It includes two functions: `area` which calculates the area of a rectangle, and `perim` which calculates the perimeter. The `main` function creates a `rect` object with width 10 and height 5, prints its area and perimeter, and then demonstrates passing a pointer to the `rect` object to the `area` and `perim` functions. The IDE interface includes a sidebar with Explorer, Timeline, and a terminal at the bottom.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Praticar Gerenciamento de dependências em Go